

Hybrid Risk-Adjusted Pricing and High-Performance Quantitative Valuation Models

Dean Foulds

December 31, 2025

Abstract

This study presents a Hybrid Risk-Adjusted Pricing Model designed to improve the accuracy and interpretability of sale price predictions for surplus and life-limited aircraft parts, combined with a high-performance quantitative asset valuation model using JAX. The approach employs a multi-stage architecture:

Phase 1 - Phase 1 establishes the foundation for developing a machine learning model to predict aircraft part pricing. This phase focuses on collecting data from internal systems (sales records, inventory, CRM, supplier costs) and external sources (industry databases, competitor pricing, OEM catalogues, market reports). Key data includes part specifications, pricing history, market conditions, and aircraft compatibility. The process involves automated data collection pipelines, rigorous cleaning procedures to handle missing values and outliers, and standardisation across sources. Exploratory data analysis identifies pricing patterns, correlations, and key variables influencing costs. Deliverables include a cleaned dataset of 10,000+ parts, data quality audit, EDA report with visualisations, and preliminary feature importance analysis. Success requires 85%+ data quality across variables and clear identification of pricing determinants. The 16-week timeline covers data sourcing, pipeline development, cleaning, analysis, and stakeholder review, culminating in approval to proceed to Phase 2:

Phase 2 - Following Phase 1's data collection and analysis, Phase 2 focuses on developing a proof of concept over 8-12 weeks to test and validate pricing prediction models. This phase involves selecting appropriate machine learning algorithms (regression models, ensemble methods, neural networks), training models on the curated dataset, and iteratively refining features and hyperparameters. Key activities include comparing model-generated prices against existing pricing from procurement teams, supplier quotes, and competitor benchmarks to assess accuracy and reliability. The testing phase identifies gaps in data coverage, prompting collection of additional variables as needed to improve predictions. Model performance is evaluated using metrics such as mean absolute error, R-squared values, and business-relevant thresholds. Deliverables include a working prototype, performance comparison reports, identified data gaps with collection plans, and recommendations for model improvements. Success requires achieving pricing predictions within 15% accuracy for 80% of parts compared to current methods. This proof of concept determines viability before full-scale deployment in Phase 3.

Phase 3 -Following successful proof of concept validation in Phase 2, Phase 3 involves comprehensive model development and integration over an estimated 20 weeks. This phase tests the refined model against large-scale, known, and pre-processed datasets

to ensure robustness and accuracy across diverse aircraft part categories and market conditions. Activities include final model optimisation, extensive validation testing, performance benchmarking at scale, and development of the production-grade prediction application. The application specifications are to be determined based on business requirements, potentially including web interface, API integration, or standalone software. Key deliverables encompass the finalised ML model, comprehensive testing reports demonstrating accuracy across the full dataset, integrated application for running predictions, user documentation and training materials, and deployment plan for operational rollout. Success criteria include maintaining prediction accuracy thresholds on the expanded dataset and seamless integration with existing business systems.

Contents

1	Introduction	5
1.1	Background and Motivation	5
1.2	Problem Statement	5
1.3	Research Questions/Objectives	5
1.4	Hypothesis	5
2	Literature Review	6
2.1	Related Work	6
2.2	Theoretical Foundation	6
3	Data Acquisition and Understanding	7
3.1	Data Sources and Limitations	7
3.2	Data Description and Forward Vision	7
3.3	Data Collection Methodology	9
4	Premium FMV Pricing Data Overview	9
4.1	Sample ILS Derived Data	9
5	Exploratory Data Analysis (EDA)	9
5.1	Statistical Summary	9
5.2	Data Visualization	10
5.3	Key Observations	12
6	Data Pre-processing For Machine Learning	12
6.1	Data Cleaning	12
6.2	Polars and not pandas	13
6.3	Data Transformation	14
6.4	Outlier Filtering	14
6.5	Handling Imbalanced Data	15
7	Feature Engineering	15
7.1	Feature Selection	15
7.2	Feature Creation	16
7.3	Final Feature Set	17

8	Methodology	18
8.1	Experimental Design	18
8.2	Conceptual Risk Model for Parts Pricing	18
8.3	Risk Calculation Example: Aircraft Turbine Blade	20
8.3.1	Worked Example: Turbine Blade After 5000 Hours	20
8.4	JAX Model Implementation	21
8.4.1	Network Structure	22
8.4.2	Parameter Initialisation	22
8.5	Accelerated Training Process	22
8.5.1	Loss Function and Gradient Calculation	22
8.5.2	JIT Compilation	23
8.5.3	Optimisation with Optax	23
8.5.4	Evaluation Criteria	23
8.5.5	Model Improvements	23
8.6	Data Splitting Strategy	24
8.7	Model Selection	24
8.8	Evaluation Metrics	24
9	Model Training Process	24
10	Model 1: Light GBM	25
11	Model 2: LSTM	25
11.1	LSTM Forecasting Pipeline	26
11.2	Windowing methodology:	26
11.2.1	LSTM Model Architecture	26
11.3	Regularisation Techniques	26
11.4	Model Interpretation	29
11.5	Results and Model Evaluation	29
11.6	Hyperparameter Tuning	29
12	Model 3 Stochastic Variational Inference (SVI)	30
13	Model 4 Markov Chain Monte Carlo (MCMC)	30
13.1	Posterior Distribution	30
13.2	Hamiltonian Monte Carlo (HMC)	31
13.2.1	Potential and Kinetic Energy	31
13.3	The Mass Matrix	31
13.4	Leapfrog Integration	32
13.5	Metropolis Acceptance Step	32
13.6	No-U-Turn Sampler (NUTS)	33
13.7	Hyperparameters	33
13.8	Final Model Selection	33
13.9	Test Set Performance	34
14	Model Training and Development	34
14.1	Baseline Model	34
14.2	Error Analysis	34

15 Predictions and Deployment	34
15.1 Making Predictions	34
15.2 Model Deployment Strategy	34
15.3 Scalability Considerations	34
16 Model Training Results	34
17 Discussion	34
17.1 Interpretation of Results	34
17.2 Comparison with Hypothesis	34
17.3 Limitations	35
17.4 Potential Biases	35

1 Introduction

1.1 Background and Motivation

The global market for surplus and used aircraft parts is volatile and highly sensitive to component condition. For high-value, life-limited parts—such as those tracked by Time Since New (TSN) and Cycle Since Repair (CSR)—mispricing due to inaccurate risk assessment can lead to significant financial losses or regulatory issues. Current pricing models often rely on generalised historical averages, failing to account for the unique operational stress and remaining useful life (RUL) of individual components. A reliable, objective system is needed to accurately convert complex, domain-specific operational data into a quantifiable risk factor. This project is motivated by the necessity to enhance market efficiency and transparency, ensuring that part pricing directly reflects its true liability and residual value.

1.2 Problem Statement

The central problem is the lack of a standardised, quantitative method to translate highly specific operational data (e.g., Cycle Since New, Time Life) of an aircraft part into a measurable risk feature for use in price prediction. Specifically, the challenge involves: (1) developing an interpretable mathematical model, implemented using JAX, to consolidate multiple operational metrics into a single, bounded Total Risk Score (R); (2) integrating this score as a vital engineered feature into an XGBoost regression model and other ML frameworks; and (3) demonstrating that the hybrid architecture yields superior price prediction accuracy compared to models that exclude this calculated risk. The ultimate goal is to solve the ambiguity in pricing by systematically accounting for the component's inherent operational risk.

1.3 Research Questions/Objectives

- **Objective 1 (Phase 1):** To successfully implement and optimise a high-performance, differentiable Conceptual Risk Model (CRM) using JAX that translates complex operational metrics into a single Total Risk Score (R).
- **Objective 2 (Phase 2):** To engineer and calculate a robust Composite APVI using financial time-series data to represent the external economic risk factor.
- **Objective 3 (Phase 3):** To train and validate time series regression models using the generated Risk Score (R) as a primary feature to achieve superior price prediction accuracy on the held-out test dataset.
- **Objective 4 (Phase 3-4):** To document the final model architecture and provide a framework for the governance and interpretability of pricing decisions based on the derived risk scores and feature contributions.

1.4 Hypothesis

The hypothesis is that the multi-stage hybrid architecture—which incorporates a rigorously engineered Total Risk Score (R) derived from the JAX Conceptual Risk Model into an XGBoost or similar regression framework—will achieve a statistically significant improvement in price prediction accuracy (as measured by a lower Root Mean Square Error (RMSE) and

Mean Absolute Error (MAE)) compared to a control model that only uses the operational features directly. Furthermore, we expect the optimised weights in the JAX model to show that the life-cycle factors (Total Time Life and Cycle Since New) are the most influential predictors of intrinsic risk.

2 Literature Review

2.1 Related Work

A survey of existing solutions and research related to pricing and risk modelling in aviation parts markets, machine learning approaches to asset valuation, and applications of gradient-based optimisation in financial modelling should be structured around the following core themes:

- **Aviation Parts Pricing and Asset Valuation:** Traditional statistical approaches, such as Hedonic Pricing Models (HPMs), are crucial for initial valuation, decomposing an asset's price based on key technical characteristics (e.g., age, condition, cycles, ownership history). This area also includes specialised research into Life-Limited Parts (LLP) Valuation, focusing on the non-linear economic depreciation and Remaining Useful Life (RUL) of high-stress components tracked by metrics like Time Since New (TSN) and Cycle Since New (CSN).
- **Machine Learning (ML) in Asset Valuation:** This strand covers the application of advanced ensemble methods like XGBoost and Random Forest for robust price prediction, particularly where data is structured and exhibits complex, non-linear relationships. Furthermore, the use of deep learning, specifically Long Short-Term Memory (LSTM) networks, is necessary to capture complex temporal dependencies and volatility patterns inherent in market price time series data.
- **High-Performance Computing and Gradient-Based Optimisation:** The literature on modern computational finance emphasises the use of frameworks like JAX for implementing custom risk and valuation functions that require high speed and scalability. Research in this area focuses on leveraging Automatic Differentiation (AutoDiff) and Just-In-Time (JIT) compilation to build and optimise complex, bespoke financial and risk modelling components (such as the Conceptual Risk Model) with maximum computational efficiency.

The following are links to the examples of NumPyro in action on time series data:

- [Time Series forecasting with NumPyro](#)
- [Git Hub Repo for NumPyro](#)

2.2 Theoretical Foundation

Data collection formed the most important and often difficult task due to a lack of information available. Quantum fields required for Hedonistic analysis are rarely populated such as time since new, cycle since new, time since repair, cycles since repair, cycle since inspection time life entry date to name a few are not available. Particularly the latter variable is key to creating

a Hedonic model, since this could be used to calculate depreciation of a part. Ideally all of these values would be available for any regression models such as XGBoost or Random Forest or any AI price predictor as they could be stored as vector scores. Solutions to deal with the vast level of historic part number data Polars instead Pandas, due to it's lazy loading ability. and JAX which has just in time implementation of mathematical operations. Lazy regression algorithm will be used for initial model evaluation based on a merged data set of Snowflake derived data.

3 Data Acquisition and Understanding

3.1 Data Sources and Limitations

The initial modelling effort utilised data from several internal and external sources, though each source carries specific limitations:

- **ILS Inventory Data (External):** This provides historical market availability. However, the methodology used by ILS to aggregate and present this data is unknown and opaque, making it difficult to fully assess its reliability. Crucially, ILS data reflects the broader market's inventory and is not fully representative of proprietary AWJ-Group transaction history, inventory quality, or customer base.
- **Snowflake Auto-Pricing Project (Sourced from Quantum):** This provided short-term pricing brackets (3 months). A major limitation is the known inaccuracy of the data stored in the Quantum system, which suffered from issues in data entry, consistency, and fidelity.
- **AJW Cost Management Department (Internal)** - consists of manual Excel spreadsheet outputs.
- **Sales Order Data (AJW Sales Department - YTD)** - Excel export to csv time series data based on top 200 parts initially.

3.2 Data Description and Forward Vision

Initial analysis involved merging three subsets of data—ILS exports, auto-pricing brackets, and sales quote data—into one machine-readable set for Exploratory Data Analysis (EDA). This merged set contains part numbers for our top 200 selling parts.

Recognising the fundamental limitations and lack of accuracy in the existing data (particularly from the Quantum system), the forward vision is to implement enhanced data governance and storage practices to ensure that data captured in the future is reliable, consistent, and fully representative of AWJ-Group's actual activities and financial results. This proactive approach will be essential for validating and improving the next generation of quantitative valuation models.

Samples of the initial data sets used as an experiment are shown below. The initial FMV(Fair Market Value) data set was a merge and mapping of these sets.

Table 1: Sample of auto-price data used

PN	Condition Code	Description	Quote Unit Price	Auto Pricing Bracket
3291432-2	REP	VALVE-LOAD CONTROL	\$110,000	\$106,229 – \$112,521
C19755BA01	REP	EDU-ENHANCED DISPLAY UNIT	\$130,000	\$121,122 – \$129,878
C19755BA01	REP	EDU-ENHANCED DISPLAY UNIT	\$130,000	\$121,122 – \$129,878
C19755BA01	REP	EDU-ENHANCED DISPLAY UNIT	\$130,000	\$121,122 – \$129,878
Z014H010035B	REP	DIRECTOR-CIDS	\$130,000	\$130,000 – \$130,000
3-1546	OH	MLG-WHEEL ASSEMBLY	\$19,000	\$17,134 – \$18,866
C20195162	OH	WHEEL ASSY-MLG	\$18,000	\$17,644 – \$18,106
C20195162	OH	WHEEL ASSY-MLG	\$19,000	\$17,644 – \$18,106
C20195162	OH	WHEEL ASSY-MLG	\$19,200	\$17,644 – \$18,106
C20195162	OH	WHEEL ASSY-MLG	\$15,000	\$17,644 – \$18,106
3888579-202	INS	ECB-ELECTRONIC CONTROL BOX, VERSA-TILE	\$190,000	\$185,447 – \$191,220

Table 2: Sample quote data for HG2030BE04 derived from sales data as sold

Entry Date	Quote Number	PN	Condition Code	Qty	Quote Total
01/01/2025	2694804	HG2030BE04	REP	1	\$30,000.00
01/02/2025	2695215	HG2030BE04	INS	1	\$30,000.00
01/02/2025	2695215	HG2030BE04	INS	1	\$340,000.00
01/02/2025	2695215	HG2030BE04	NE	1	\$366,656.25
01/02/2025	2695463	HG2030BE04	NE	1	\$366,656.25
01/03/2025	2696448	HG2030BE04	REP	1	\$27,500.00
01/06/2025	2697245	HG2030BE04	REP	1	\$30,000.00
01/08/2025	2699995	HG2030BE04	NE	1	\$488,875.00
01/08/2025	2699995	HG2030BE04	REP	1	\$30,000.00
01/09/2025	2701058	HG2030BE04	NE	1	\$488,875.00
01/12/2025	2702621	HG2030BE04	NE	1	\$366,656.25

3.3 Data Collection Methodology

The data sources are stored in various locations. Methods to obtain data were not limited to just database queries and the following list is not exhaustive:

- Snowflake queries were used on top 200 company part numbers based on part ranking
- Top 200 parts based on ranking sent to ILS for FMV data sourcing - returned csv
- Sales team data records - csv export
- Incoming parts documentation in paper format
- AMOS xml data dumps in AWS s3 buckets

4 Premium FMV Pricing Data Overview

The dataset analysed contains (FMV) pricing records for high-value inventory items, capturing historical quotes, predicted prices, and confidence levels.

4.1 Sample ILS Derived Data

Table 3: Fair Market Value (FMV) Data Summary

PN	Cond.	FMV	FMV Low	FMV High	Min Price	Max Price
442653	OH	\$901,315.91	\$721,052.73	\$1,081,579.09	\$150,000.00	\$1,300,000.00
442653	OH	\$640,000.00	\$512,000.00	\$768,000.00	\$640,000.00	\$640,000.00
442653	NE	\$1,467,538.83	\$1,174,031.06	\$1,761,046.60	\$1,400,000.00	\$2,080,621.28
442653	SV	\$557,943.28	\$446,354.63	\$669,531.94	\$85,000.00	\$649,000.00
772292	OH	\$75,874.77	\$60,699.81	\$91,049.72	\$63,000.00	\$100,000.00
772292	SV	\$78,332.62	\$62,666.10	\$93,999.14	\$4,500.00	\$156,000.00
772292	AR	\$94,128.57	\$75,302.86	\$112,954.29	\$200.00	\$225,000.00
772292	NE	\$439,659.18	\$351,727.34	\$527,591.01	\$245,000.00	\$800,751.35
849589	NS	\$82,784.86	\$66,227.89	\$99,341.83	\$9,500.00	\$95,000.00
849589	SV	\$35,776.56	\$28,621.24	\$42,931.87	\$28,750.00	\$45,000.00
849589	OH	\$65,277.29	\$52,221.83	\$78,332.75	\$33,480.00	\$120,000.00

5 Exploratory Data Analysis (EDA)

5.1 Statistical Summary

A merge of the 3 subsets then a break down of fiscal value areas of the business produced the following results.

5.2 Data Visualization

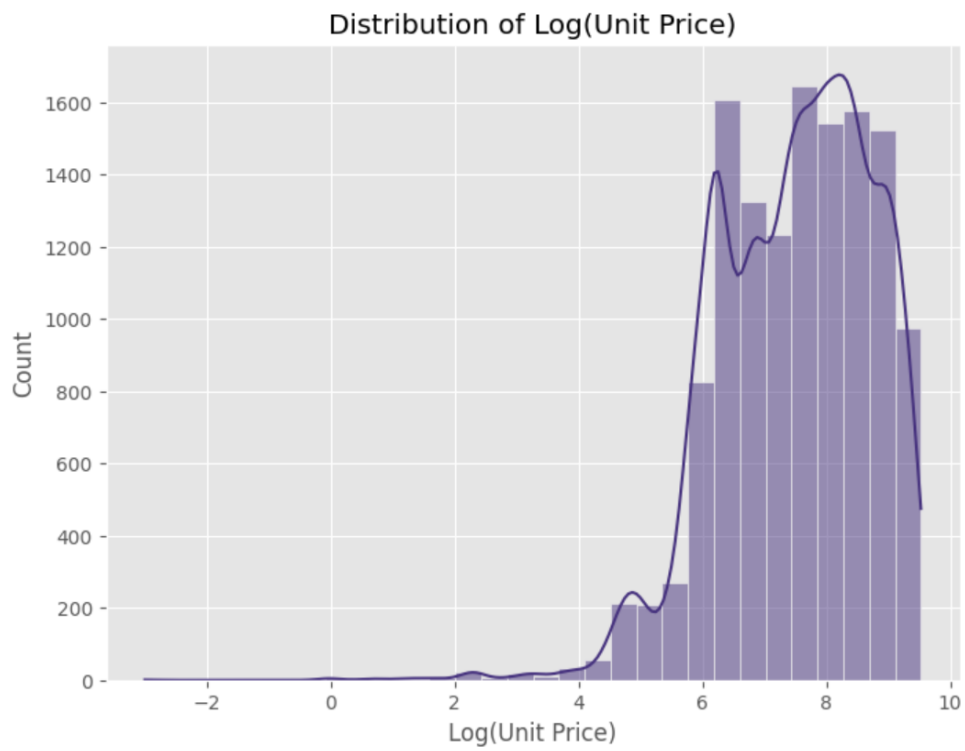


Figure 1: Initial view of all sales by price range and condition.

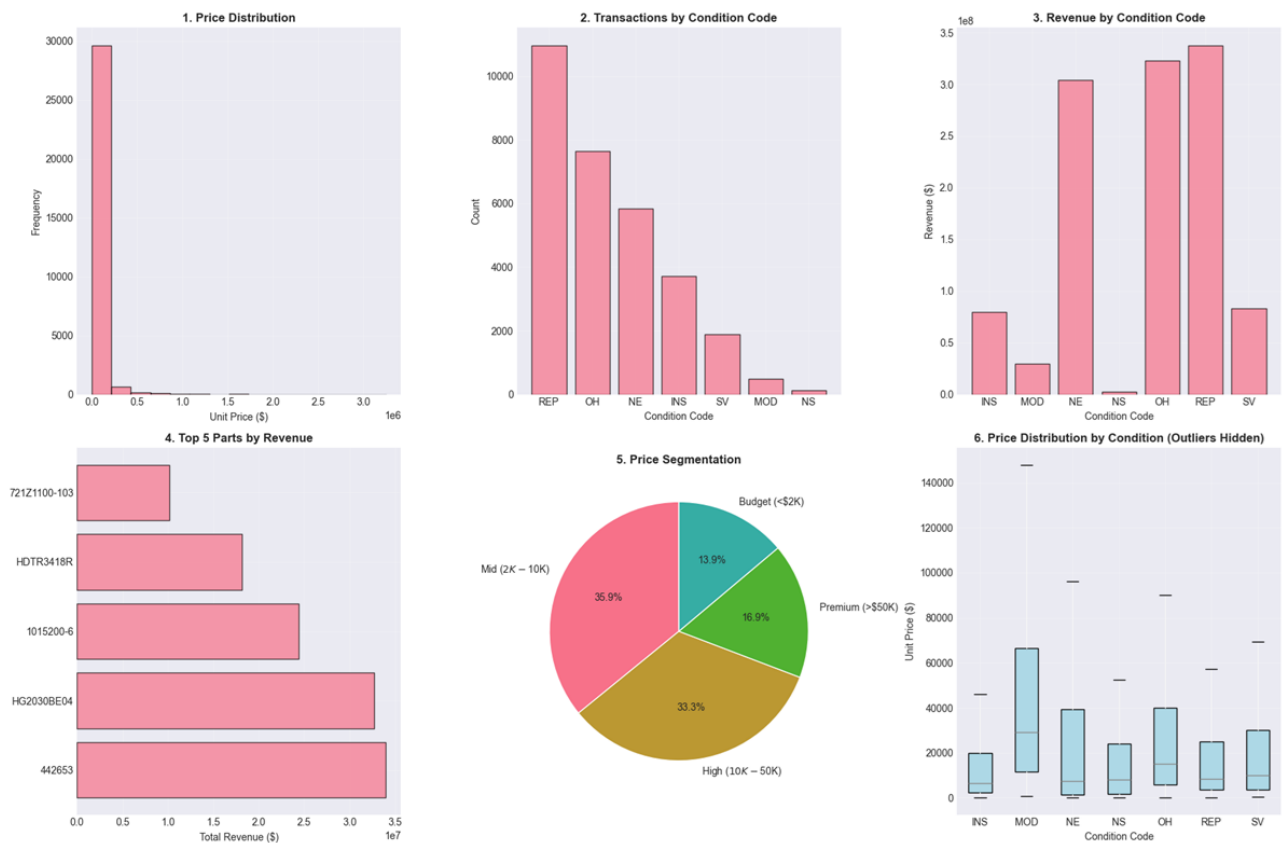


Figure 2: Initial view of all sales by price range and condition.

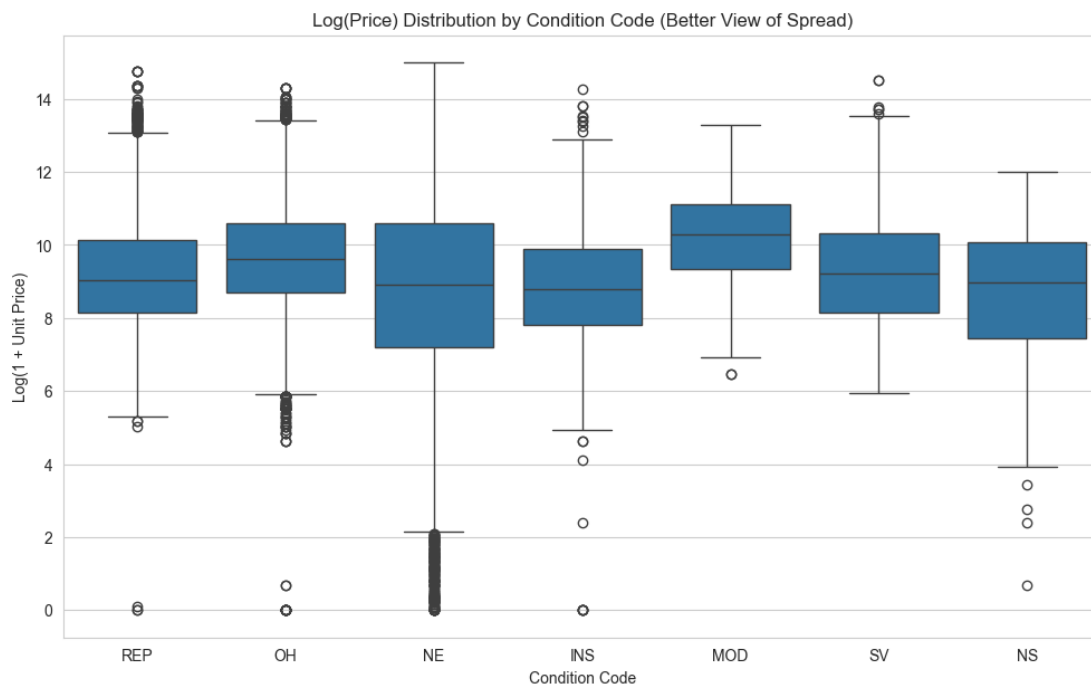


Figure 3: Better view of box plots.

5.3 Key Observations

Data does not fit normal distribution which is key to training so log needs to be applied.

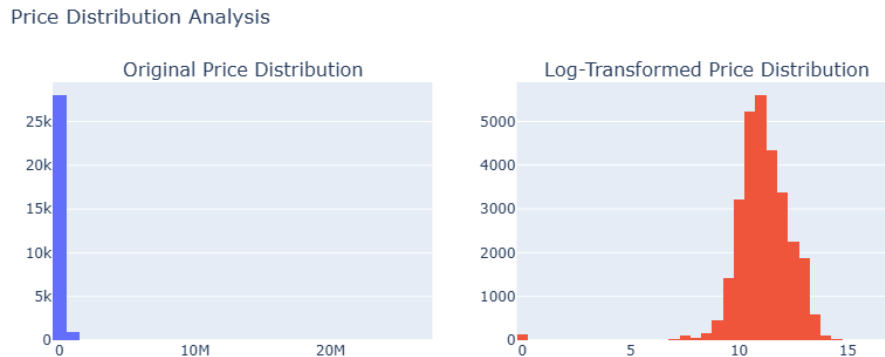


Figure 4: Initial data explore of pricing distribution.

The data is made up of INS(Inspected), OH(Overhauled), NE(New), REP(Repaired), SV(Serviceable), MOD(Modified) and NS(Non-serviceable). In terms of value distribution it can be seen that the majority, nearly 50% of the companies business revenue is from 0\$ - \$10k with a good proportion between \$10k - \$50k. The majority of condition codes NE, OH, REP and SV make up the bulk of the data which is logical due to the nature of the business.

Initial LazyRegressor tests showed best result being LGBMRegressor the best option although all regression RMSE model scores were not acceptable however the LGBMRegressor was run on a test set to see the actual error and \$92,113.23 across the top 200 parts not an acceptable value considering the overall price range so the decision to investigate a more robust ML model was taken.

Next steps run data in Vertex AI for a possible solution.

6 Data Pre-processing For Machine Learning

6.1 Data Cleaning

Mention issues with data sets and missing values.

- White spaces, "-", ",", and "\$" all removed in cleaning process prior to machine learning (see notebook code).
- Missing values: Most missing values rows dropped due to sufficient stock line data being representative.
- Outliers: sigma 3, 4 and 5 used and also for vertex training accuracy 25% and 75% percentiles to remove very unusual prices.
- Duplicates: duplicate part numbers - handled as stock lines initially, then just part number with categorical condition.
- Inconsistencies: dropping quartile outliers.

6.2 Polars and not pandas

The Polars library has emerged as a compelling alternative to Pandas for data manipulation in Python, offering several significant advantages that make it particularly attractive for modern data processing tasks.

Performance and Speed

Polars delivers substantially faster performance than Pandas, often achieving speed improvements of 5-10 times or more for common operations. This performance gain stems from its implementation in Rust and its use of Apache Arrow as its memory model, which enables more efficient data handling than Pandas' NumPy-based architecture. The library executes operations in parallel by default, automatically utilising multiple CPU cores without requiring additional configuration from the user, whereas Pandas primarily operates on a single thread unless explicitly configured otherwise.

Memory Efficiency

The memory footprint of Polars is considerably smaller than that of Pandas. The Apache Arrow columnar format stores data more compactly and allows for better cache utilisation. Additionally, Polars supports streaming datasets that exceed available RAM through its lazy evaluation mode, enabling users to process files larger than their system memory—something that proves challenging with Pandas without resorting to chunking strategies. Query Optimisation Polars implements a lazy evaluation system that doesn't execute operations immediately. Instead, it constructs a query plan that can be optimised before execution. The query optimiser automatically applies techniques such as predicate pushdown, projection pushdown, and query parallelisation. This means operations are reordered and combined to minimise computational work and memory usage, producing more efficient execution without manual intervention. Modern API Design The Polars API offers a more consistent and intuitive interface compared to Pandas. Method names and behaviours are more predictable, and the library avoids many of the historical inconsistencies that accumulated in Pandas over its development. Polars supports method chaining naturally, making data transformation pipelines more readable and maintainable. Type System and Data Integrity Polars maintains a stricter type system than Pandas, which helps catch errors earlier in the development process. Unlike Pandas, Polars doesn't silently convert data types or produce unexpected null values during operations, leading to more predictable behaviour and fewer subtle bugs in data processing pipelines. Handling Missing Data Whilst Pandas uses multiple representations for missing values (NaN, None, NaT), Polars uses a single null representation across all data types. This consistency simplifies reasoning about missing data and eliminates confusion about how different operations handle various missing value types. These advantages make Polars particularly well-suited for large-scale data processing, ETL pipelines, and scenarios where performance and memory efficiency are critical considerations. However, Pandas remains valuable for its extensive ecosystem, broader community support, and integration with numerous existing libraries and tools.

6.3 Data Transformation

- Label encoding of condition, confidence and RC (Sale or Exchange).
- One-Hot encoding of part numbers for Numpyro operations (model computational expense).
- Scaling of large integers.
- Log transformations for all model training.
- conversion of numerical values as strings values to integers and then scaled for better ML performance.
- Scaled values back to fiscal values for financial analysis.

6.4 Outlier Filtering

Critical to the integrity of the training dataset is the removal of extreme price deviations. The filtering process involves removing samples where the `quoteunitprice` falls outside a 5% tolerance of the `fmv` value. This is used to eliminate extreme outliers that could distort the model's understanding of the fair market range:

$$L_{\min} < \text{quoteunitprice} < L_{\max}$$

where $L_{\min} = 0.95 \times \text{fmv}$ and $L_{\max} = 1.05 \times \text{fmv}$.

6.5 Handling Imbalanced Data

In the all datasets there were a number of imbalances. For example a part maybe priced at \$900,000 on one occasion and \$0.05 on another occasion. To deal with this 3 thresholds were tested and a \$15,000 threshold was set to filter out these skews in the data.

It could also be seen that there were some really large outliers in the sales data a whole so to manage this, stratification was employed as a result a low, medium and high bucketing was set up for training. This improved the result and performance the most.

Top 3 PNs: Log Price vs. Time Colored by Specific Condition (Interactive)



Figure 5: Indication of differences in price given the same part condition.

7 Feature Engineering

7.1 Feature Selection

In the initial data set which was a merge of the 3 given samples is shown below:

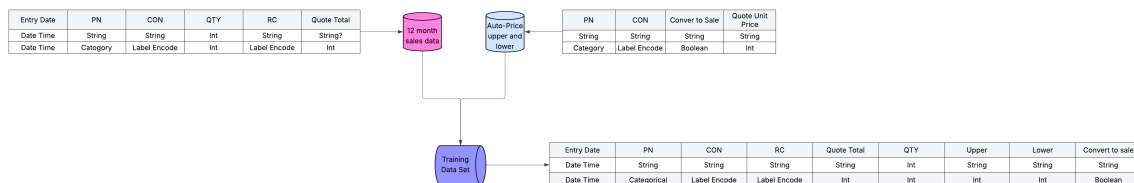


Figure 6: Merged data selection for model training without ILS data mapping.

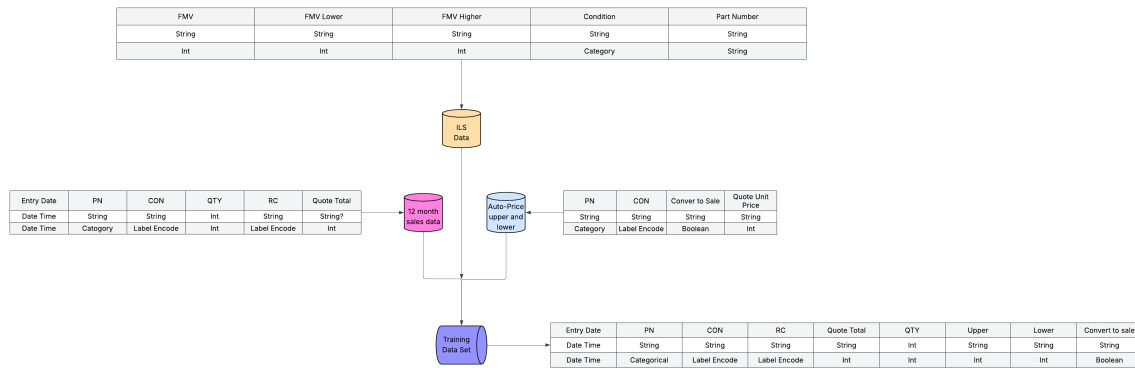


Figure 7: Merged data selection for model training with ILS data mapping.

7.2 Feature Creation

- Correlation analysis
- Feature importance from preliminary models
- Domain knowledge

The input feature vector x consists of the following components:

- **Market Dynamics:** Volatility metrics, moving averages, and correlation coefficients relative to broad market indices.
- **Intrinsic Factors:** Key financial ratios (e.g., P/E, P/B), liquidity scores, and sector-specific indicators.
- **Categorical Encoding:** One-hot encoding of asset types, industry sectors, and geographical regions.
- **Engineered Total Risk Score (JAX):** Computed from operational metrics.
- **Composite APVI feature:** Representing external economic risk.

All numerical features are standardised (zero mean, unit variance) to ensure faster convergence during training.

7.3 Final Feature Set

Sales Order	Date	RC	Part Number	CON	Qty	Unit (£)	Total (£)
ADH172215	25/09/25	S	NAIL FILES	NE	1	0.05	0.05
ADH165260	25/03/25	S	MS9592-014	NE	1	0.95	0.95
ADH172215	25/09/25	S	NAPKINS - COCKTAIL	NE	1	1.00	1.00
ADH172215	25/09/25	S	NAPKINS - DINNER	NE	1	1.00	1.00
ADH172215	25/09/25	S	TENT CARDS	NE	1	1.00	1.00
ADH173557	06/10/25	S	1146001-0100-01	NS	1	1.00	1.00
ADH165569	29/03/25	S	NSA5414-32-12	NE	1	1.79	2.31
ADH165569	29/03/25	S	ABS0718A05	NE	1	1.99	2.57
ADH165569	29/03/25	S	ABS0718-07	NE	1	1.99	2.57
ADH163493	20/02/25	S	ABS0258-8A	NE	1	2.00	2.00
ADH167874	21/05/25	S	ESC36-20	NE	1	3.00	3.00
ADH172215	25/09/25	S	BUSINESS CARD HOLDERS	NE	1	3.00	3.00
ADH159670	16/05/25	S	NAS6205L20	NE	1	3.30	4.20
ADH165569	29/03/25	S	ABS0718-03	NE	1	3.58	4.62
ADH159100	16/05/25	S	AS21919WCH07	NE	1	4.00	5.16
ADH159086	01/11/24	S	8GH005678-06	NE	1	4.06	4.06

Table 4: Sample aircraft parts transaction data, scaled but not encoded.

```

-----
Top Regression Models Comparison:
-----

```

Model	Adjusted R-Squared	R-Squared	RMSE	Time Taken
LGBMRegressor	0.21	0.21	1.63	1.41
HistGradientBoostingRegressor	0.21	0.21	1.63	5.39
ExtraTreeRegressor	0.21	0.21	1.63	0.06
XGBRegressor	0.21	0.21	1.63	0.88
DecisionTreeRegressor	0.21	0.21	1.63	0.06
RandomForestRegressor	0.21	0.21	1.63	5.49
ExtraTreesRegressor	0.21	0.21	1.64	1.51
BaggingRegressor	0.21	0.21	1.64	0.28
GradientBoostingRegressor	0.19	0.19	1.65	4.08
GaussianProcessRegressor	0.17	0.18	1.67	174.94
MLPRegressor	0.17	0.17	1.68	5.57
SVR	0.13	0.14	1.71	86.67
NuSVR	0.13	0.13	1.72	26.41
...				

Figure 8: Initial regression tests on the above data set.

```

-----
Model: LGBMRegressor
Root Mean Squared Error (RMSE) on Test Set: $92,113.23
R-squared (R2) Score on Test Set (Log Scale): 0.2424
-----

```

Figure 9: Initial regression tests.

8 Methodology

8.1 Experimental Design

This section details the full experimental pipeline, from data preparation to model evaluation. The approach combines mathematical rigor, reproducibility, and a clear explanation of how each computational component contributes to the final predictions.

8.2 Conceptual Risk Model for Parts Pricing

To accurately price used or surplus aircraft parts, it is essential to incorporate a quantitative assessment of the associated risk. This risk is primarily driven by factors such as a part's **traceability, age, condition, current market volatility** and **storage history**. A **Total Risk Score** (R) is defined using a weighted linear combination of normalised risk factors, passed through a sigmoid function to ensure the score is bounded between 0 and 1. The mathematical formulation for the Total Risk Score (R) is:

$$R = \sigma \left(\sum_{i=1}^n w_i \cdot X_i - T \right) \quad (1)$$

Where:

- R : The **Total Risk Score**, a value in the range $[0, 1]$.
- $\sigma(\cdot)$: The **Sigmoid Function**, defined as $\sigma(z) = \frac{1}{1+e^{-z}}$.
- X_i : The i -th **Normalised Risk Factor Score**, with $X_i \in [0, 1]$.
- w_i : The **Weight** applied to the i - variables vectored scoring for sigmoid function.
- T : A **Threshold** (or bias) parameter - Hedonic depreciation calculation.

This risk score R is directly proportional to the perceived liability or uncertainty of the part. The use of **JAX** allows for efficient calculation of gradients, enabling rapid optimisation of the parameters (w_i and T) against historical data.

Relevance of the Sigmoid Function in Bounding Risk Scores

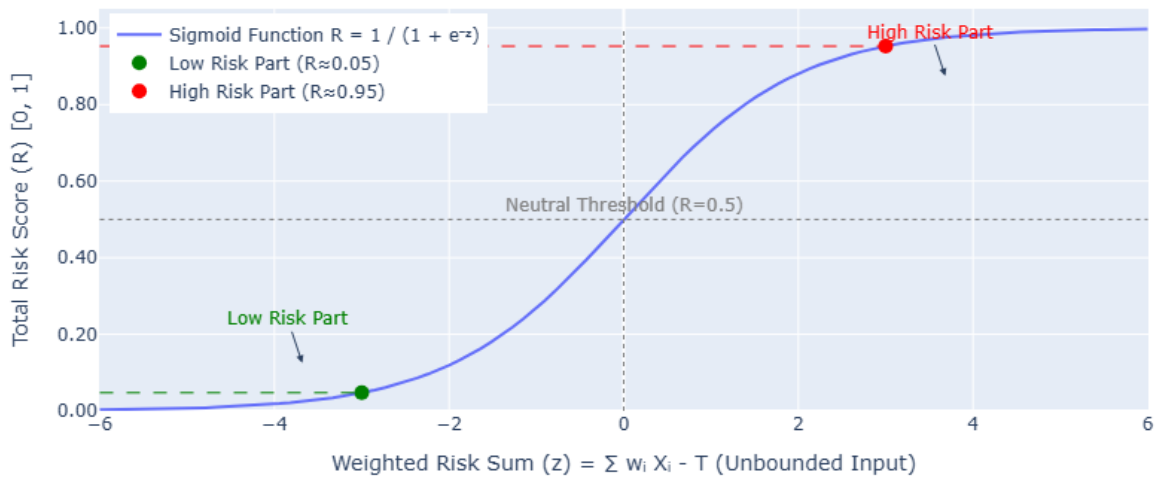


Figure 10: Visualising the function.

An example of a headonic (T) aerospace could be:

Hedonic Price Model:

$$P_t = \beta_0 + \sum_{i=1}^n \beta_i X_{i,t} + \epsilon_t$$

Where:

- P_t = Price at time t
- $X_{i,t}$ = Value of characteristic i at time t
- β_i = Implicit price of characteristic i
- ϵ_t = Error term

Component: Aircraft Turbine Blade

Characteristic	New (t=0)	After 5000 hrs (t=1)	β (£)
Remaining cycles	10,000	5,000	£50/cycle
Coating thickness (mm)	2.0	1.2	£25,000/mm
Crack-free status	1 (yes)	1 (yes)	£100,000
Certification valid	1 (yes)	1 (yes)	£50,000

Characteristics & Weights (β coefficients from regression):

Depreciation Rate:

$$D_t = \frac{P_0 - P_t}{P_0} = 1 - \frac{\sum_{i=1}^n \beta_i X_{i,t}}{\sum_{i=1}^n \beta_i X_{i,0}}$$

$$P_0 = 50 \times 10,000 + 25,000 \times 2.0 + 100,000 + 50,000 = \text{£}700,000$$

$$P_1 = 50 \times 5,000 + 25,000 \times 1.2 + 100,000 + 50,000 = \text{£}430,000$$

$$D_1 = \frac{700,000 - 430,000}{700,000} = 0.386 \text{ or } 38.6\% \text{ depreciation}$$

8.3 Risk Calculation Example: Aircraft Turbine Blade

Risk Score Formula:

$$R = \sigma \left(\sum_{i=1}^n w_i X_i - T \right) = \frac{1}{1 + e^{-(\sum_{i=1}^n w_i X_i - T)}} \quad (2)$$

Where:

- R : The **Total Risk Score**, a value in the range $[0, 1]$.
- $\sigma(\cdot)$: The **Sigmoid Function**, defined as $\sigma(z) = \frac{1}{1+e^{-z}}$.
- X_i : The i -th **Normalised Risk Factor Score**, with $X_i \in [0, 1]$.
- w_i : The **Weight** applied to the i -th variable for sigmoid function.
- T : A **Threshold** (or bias) parameter derived from hedonic depreciation calculation.

This risk score R is directly proportional to the perceived liability or uncertainty of the part. The use of **JAX** allows for efficient calculation of gradients, enabling rapid optimisation of the parameters (w_i and T) against historical data.

8.3.1 Worked Example: Turbine Blade After 5000 Hours

Step 1: Calculate Hedonic Depreciation

From our hedonic model:

$$P_0 = 50 \times 10,000 + 25,000 \times 2.0 + 100,000 + 50,000 = \text{£}700,000 \quad (3)$$

$$P_1 = 50 \times 5,000 + 25,000 \times 1.2 + 100,000 + 50,000 = \text{£}430,000 \quad (4)$$

$$D_1 = \frac{700,000 - 430,000}{700,000} = 0.386 \text{ (38.6\% depreciation)} \quad (5)$$

Step 2: Define Risk Factors and Normalise

Step 3: Calculate Weighted Sum

$$\begin{aligned} \sum_{i=1}^5 w_i X_i &= (2.5 \times 0.50) + (3.0 \times 0.40) + (1.5 \times 0.25) \\ &\quad + (2.0 \times 0.386) + (1.0 \times 0.13) \end{aligned} \quad (6)$$

$$= 1.25 + 1.20 + 0.375 + 0.772 + 0.13 \quad (7)$$

$$= 3.727 \quad (8)$$

Risk Factor	Raw Value	Normalised (X_i)	Weight (w_i)
Cycle depletion	50% used	0.50	2.5
Coating degradation	40% loss	0.40	3.0
Age (operating hours)	5000/20000	0.25	1.5
Depreciation rate	38.6%	0.386	2.0
Market volatility (APVI)	0.13	0.13	1.0

Table 5: Risk factors for turbine blade assessment

Step 4: Apply Threshold and Sigmoid Function

Assuming a threshold parameter $T = 2.0$ (calibrated from historical data):

$$z = \sum_{i=1}^5 w_i X_i - T = 3.727 - 2.0 = 1.727 \quad (9)$$

$$R = \sigma(1.727) = \frac{1}{1 + e^{-1.727}} \quad (10)$$

$$= \frac{1}{1 + 0.178} \quad (11)$$

$$= \frac{1}{1.178} \quad (12)$$

$$= 0.849 \quad (13)$$

Interpretation:

The turbine blade has a **risk score of 0.849** (or 84.9%), indicating high uncertainty in pricing and availability. This suggests:

- Significant price volatility expected
- Higher markup may be justified to account for risk
- Priority for inventory monitoring and supplier negotiations
- Potential need for alternative sourcing strategies

Risk-Adjusted Pricing:

The final price could incorporate this risk score:

$$P_{\text{adjusted}} = P_1 \times (1 + \alpha \cdot R) \quad (14)$$

Where α is the risk premium factor (e.g., $\alpha = 0.15$ for 15% maximum markup):

$$P_{\text{adjusted}} = £430,000 \times (1 + 0.15 \times 0.849) = £430,000 \times 1.127 = £484,610 \quad (15)$$

8.4 JAX Model Implementation

The valuation model is implemented as a fully connected Deep Neural Network (DNN) using the Flax library built on JAX.

8.4.1 Network Structure

The network is structured as follows:

- **Input Layer:** Matches the dimensionality of the feature vector, D_{in} .
- **Hidden Layers:** Three hidden layers with 128, 64, and 32 units.
- **Activation:** Swish activation function: $\text{Swish}(z) = z \cdot \sigma(z)$.
- **Output Layer:** Single output unit predicting FMV.

8.4.2 Parameter Initialisation

Weights are initialised using He initialisation to maintain variance stability across layers.

Listing 1: JAX Valuation Model (Flax)

```
1 import jax
2 import flax.linen as nn
3 from jax import numpy as jnp
4
5 class ValuationModel(nn.Module):
6     @nn.compact
7     def call(self, x):
8         x = nn.Dense(features=128)(x)
9         x = nn.swish(x)
10        x = nn.Dense(features=64)(x)
11        x = nn.swish(x)
12        x = nn.Dense(features=32)(x)
13        x = nn.swish(x)
14        x = nn.Dense(features=1)(x)
15        return jnp.squeeze(x)
16
17 key = jax.random.PRNGKey(42)
18 key, initkey = jax.random.split(key)
19 Din = 10
20 model = ValuationModel()
21 params = model.init(initkey, jnp.ones([1, Din]))['params']
```

8.5 Accelerated Training Process

JAX's strengths are fully leveraged through `grad` for automatic differentiation and `jit` for JIT compilation.

8.5.1 Loss Function and Gradient Calculation

The objective is to minimise Root Mean Squared Error (RMSE):

The Root Mean Squared Error (RMSE) is a frequently used measure of the differences between values predicted by a model and the actual values observed.

Brief Explanation RMSE represents the standard deviation of the prediction errors (or residuals). It tells you how concentrated the data is around the best fit line. Because the

errors are squared before being averaged, large errors are weighted more heavily than small errors.

The result of RMSE is in the same units as the target variable making it highly interpretable and directly comparable to the scale of the original data. A lower RMSE indicates a better-fitting model.

$$\text{RMSE}(\theta) = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}$$

8.5.2 JIT Compilation

JIT compilation compiles functions into optimised XLA computation graphs, which eliminates Python overhead.

8.5.3 Optimisation with Optax

Optax Integration: State-of-the-art optimisers like Adam are used for parameter updates.

Vectorisation: JAX uses `vmap` to vectorize operations over batch dimensions.

Predicted values are inverse-scaled and exponentiated:

$$\hat{F} = \exp(\hat{y}_{\log}) = \exp(\tilde{y} \cdot \sigma_{\log} + \mu_{\log})$$

Approximate dollar-scale MAE:

$$\widehat{\text{Dollar MAE}} \approx \bar{F} \cdot (e^{L\sigma_{\log}} - 1)$$

8.5.4 Evaluation Criteria

- **Good Scoring:** MAE < 5% of average FMV
- **Moderate Scoring:** MAE between 5% and 15%
- **Poor Scoring:** MAE > 15%

8.5.5 Model Improvements

Data Improvements:

- Introduce richer operational or market variables
- Remove or adjust outliers
- Data augmentation or hierarchical grouping

Model Improvements:

- Test Transformer or TCN architectures
- Hyperparameter optimisation
- Probabilistic or quantile regression

8.6 Data Splitting Strategy

Initial strategy:

- Training set: 70-80%
- Validation set: 10-15%
- Test set: 10-15%

For smaller datasets and LSTM, cross-validation with rolling windows is used.

8.7 Model Selection

The problem is defined as a regression problem. Models tested:

1. Random Forest - baseline
2. Light GBM - complex patterns
3. LSTM - time series patterns
4. JAX/Flax Neural Network - high-performance valuation

8.8 Evaluation Metrics

- Primary metric: RMSE
- Secondary metrics: R2, MAE, MAPE

9 Model Training Process

Each model was run and scored using the Lazyregressor algorithm and scoring compared, then the highest scoring model was used on the data set and results output in a notebook cell.

10 Model 1: Light GBM

```
=====
3. MODEL TRAINING AND EVALUATION
=====
Starting LGBMRegressor training...
[LightGBM] [Warning] Found whitespace in feature_names, replace with underlines
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.004679 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 261
[LightGBM] [Info] Number of data points in the train set: 19520, number of used features: 106
[LightGBM] [Info] Start training from score 9.204728
Training complete.
-----
Model: LGBMRegressor
Root Mean Squared Error (RMSE) on Test Set: $92,113.23
R-squared (R2) Score on Test Set (Log Scale): 0.2424
-----
```

Figure 11: Post lazy regressor test.

11 Model 2: LSTM

An LSTM (Long Short-Term Memory) model is a specialised type of recurrent neural network designed to learn patterns in sequential data whilst overcoming the limitations of traditional neural networks when processing long sequences.

The fundamental challenge that LSTMs address is the difficulty standard networks face in retaining information over extended periods. When processing sequences like sentences or time series data, earlier information often becomes degraded or "forgotten" as the network processes subsequent elements - a problem known as the vanishing gradient problem. LSTMs solve this through a sophisticated internal architecture that allows them to selectively remember or forget information.

At the core of an LSTM is a memory cell that maintains its state over time, regulated by three specialised gates. The forget gate determines which information from the previous state should be discarded. The input gate controls what new information should be stored in the cell state. Finally, the output gate decides what information from the cell state should be passed to the next stage. These gates use sigmoid activation functions to produce values between 0 and 1, effectively acting as filters that can completely block information (0) or allow it through entirely (1).

This gating mechanism enables LSTMs to maintain relevant information across many time steps whilst discarding irrelevant details. For instance, when processing text, an LSTM can remember the subject of a sentence even when that subject appeared many words earlier, allowing it to maintain grammatical agreement and contextual understanding throughout long passages.

LSTMs have proved particularly effective for tasks involving sequential dependencies, including language translation, speech recognition, time series forecasting, and text generation. Their ability to capture both short-term patterns and long-term dependencies makes them considerably more powerful than simpler recurrent architectures for complex sequence modelling tasks.

11.1 LSTM Forecasting Pipeline

The LSTM pipeline processes multivariate time-series data through data cleaning, windowing, standardisation, model training, and evaluation.

Mathematical Description

11.2 Windowing methodology:

- $x_{p,t} \in \mathbb{R}^d$ represent the feature vector for product p at time t
- $F_{p,t}$ denote the observed Fair Market Value (FMV)
- $y_{p,t} = \log(F_{p,t} + 1)$ denote the log-transformed target

Sliding window of size $C = 14$ forms the input sequence:

$$X^{(i)} = [x_{p,t-C+1}, \dots, x_{p,t}]^\top \in \mathbb{R}^{C \times d}$$

Forecast horizon of $H = 14$ steps forms the target sequence:

$$Y^{(i)} = [y_{p,t+1}, \dots, y_{p,t+H}]^\top$$

StandardScaler transformation:

$$\mathcal{S}(z) = \frac{z - \mu}{\sigma}$$

11.2.1 LSTM Model Architecture

- **LSTM layer:** 128 units, ReLU activation
- **Dropout layer:** rate = 0.2
- **Dense layer:** 64 ReLU units
- **Output layer:** H continuous forecasts

11.3 Regularisation Techniques

The high-performance LSTM modelling pipeline incorporates several modern regularisation techniques to mitigate overfitting, which is critical when dealing with complex time series data:

- **Dropout:** Implemented within the `LSTMModel` class with a rate of 0.3 (`dropoutrate: float = 0.3`). This randomly ignores a fraction of neurons during training, preventing complex co-adaptations and enforcing a more robust model.
- **Early Stopping:** This technique monitors the performance metric (Validation Loss, `val_loss`) and halts training if no improvement is observed for a defined number of epochs (`patience=6`). This ensures the model saves the parameters that yield the best performance on unseen data.

- **Learning Rate Scheduling:** The cosine decay learning rate schedule is used with the Adam optimiser. While not a direct L1/L2 penalty, this controlled decay helps the model converge smoothly towards better minima and can act as an effective regulariser by preventing large, erratic parameter updates late in training.

The code also uses the Adam optimiser, which inherently performs a form of L2 regularisation when decay is applied to the learning rate.



Figure 12: Removing outliers and out of range values.

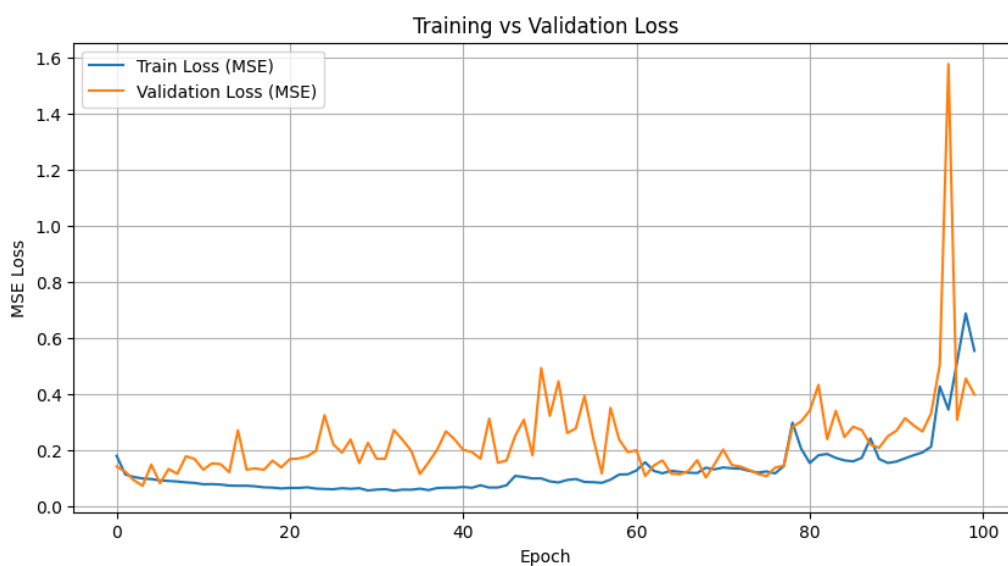


Figure 13: First LSTM results at 100 epochs.

It can be seen that the validation and test set diverged around 60-65 epochs so the model was retrained to use 62 epochs and the results are shown next.

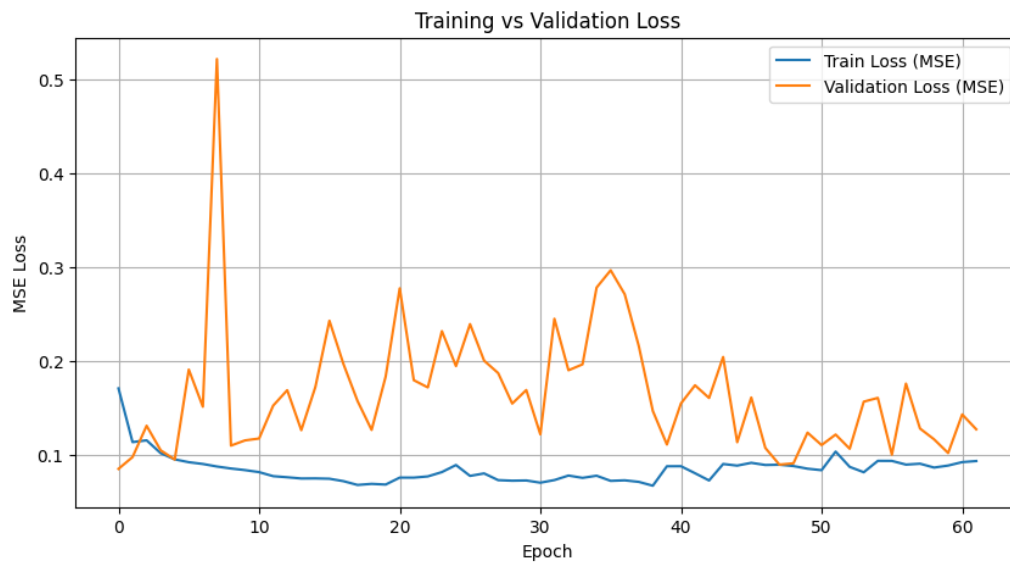


Figure 14: LSTM results at 62 epochs.

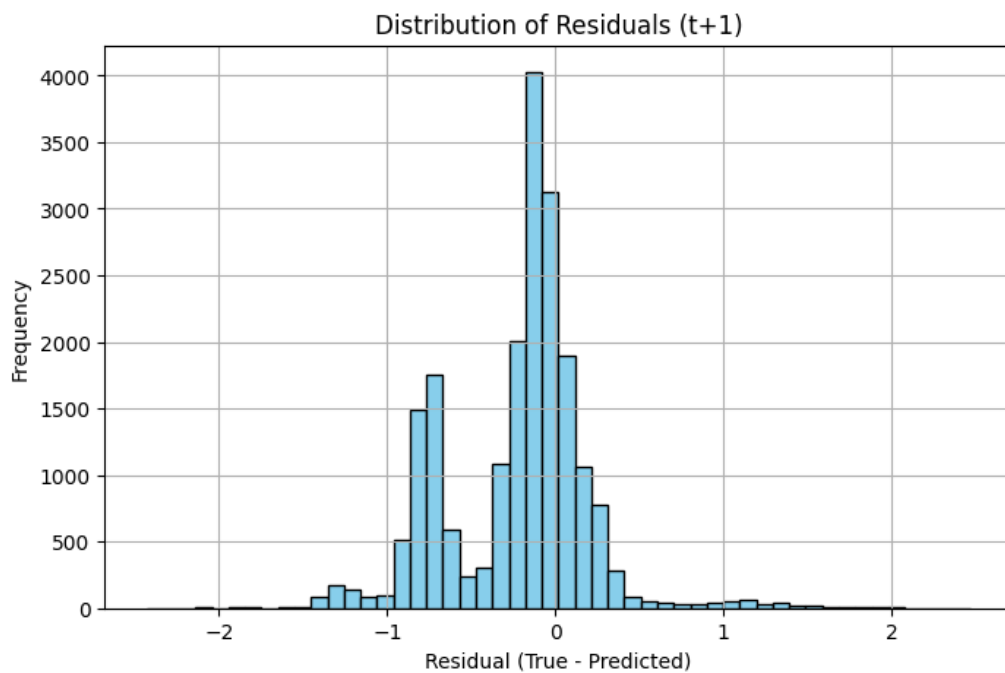


Figure 15: Distribution of residuals.

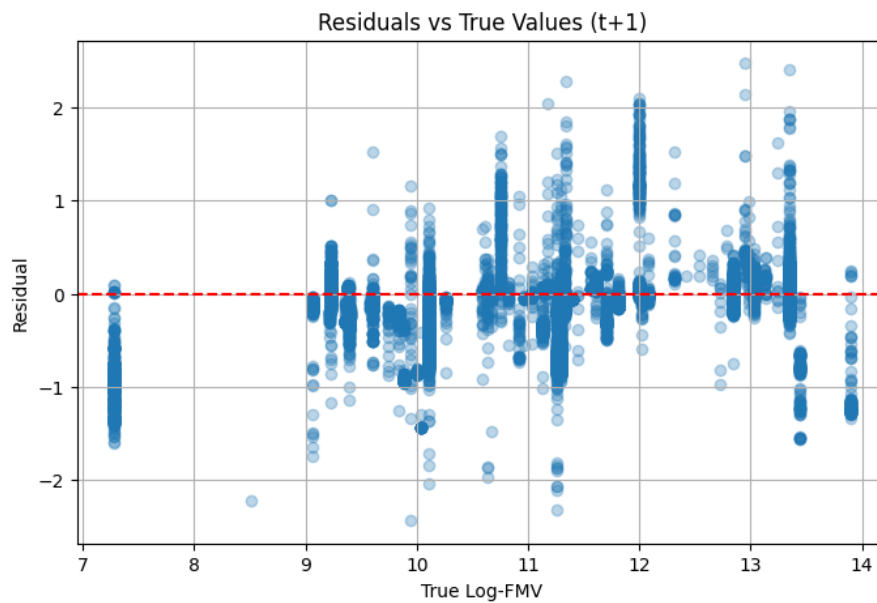


Figure 16: Test residuals for 62 epoch predictions.

Although the validation and test set diverge quite well it can be seen that it is not smooth so the data set has some erratic data in the validation set.

11.4 Model Interpretation

- Feature importance
- Decision boundaries
- Example predictions

11.5 Results and Model Evaluation

```
-----
Model: LGBMRegressor
Root Mean Squared Error (RMSE) on Test Set: $92,113.23
R-squared (R2) Score on Test Set (Log Scale): 0.2424
-----
```

Figure 17: Initial regression tests.

11.6 Hyperparameter Tuning

Windowing model implement but ineffective in reducing RMSE.

Table 6: Hyperparameter Search Space

Parameter	Range/Values	Optimal Value
Learning rate	[0.001, 0.01, 0.1]	[value]
Batch size	[16, 32, 64]	[value]
...	...	...

12 Model 3 Stochastic Variational Inference (SVI)

SVI is a fast method for approximating complex probability distributions. Instead of exact calculations, it finds a simpler distribution that closely matches the target. Think of it as fitting a smooth curve to data points.

The core idea: minimize the difference between our approximation $q(\theta)$ and the true distribution $p(\theta|data)$:

$$\text{minimize: } \text{KL}(q(\theta) \| p(\theta|data)) \quad (16)$$

SVI updates parameters iteratively using gradients:

$$\phi_{t+1} = \phi_t - \alpha \nabla_{\phi} \text{ELBO}(\phi) \quad (17)$$

where α is the learning rate and ELBO measures approximation quality.

Table 7: SVI Hyperparameters

Parameter	Value
Number of SVI Iterations	1000
Learning Rate	0.01

13 Model 4 Markov Chain Monte Carlo (MCMC)

Markov Chain Monte Carlo (MCMC) methods are used to draw samples from a target distribution that is difficult to compute directly. In Bayesian inference, this target distribution is typically the posterior distribution

$$p(\theta \mid X, y).$$

13.1 Posterior Distribution

The posterior distribution is proportional to the product of the likelihood and the prior:

$$p(\theta \mid X, y) \propto p(y \mid X, \theta) p(\theta).$$

MCMC methods do not compute this distribution analytically; instead, they construct a Markov chain whose stationary distribution *is* the posterior.

13.2 Hamiltonian Monte Carlo (HMC)

Hamiltonian Monte Carlo augments parameters θ with an auxiliary momentum variable r . A Hamiltonian function is defined as

$$H(\theta, r) = U(\theta) + K(r),$$

where:

- $U(\theta)$ is the potential energy,
- $K(r)$ is the kinetic energy.

13.2.1 Potential and Kinetic Energy

The potential energy is defined from the posterior as

$$U(\theta) = -\log p(\theta \mid X, y),$$

which shapes the “energy landscape” of the model.

The kinetic energy is defined as

$$K(r) = \frac{1}{2} r^\top M^{-1} r,$$

where M is a mass matrix (often the identity).

13.3 The Mass Matrix

In Hamiltonian Monte Carlo, we use ideas from physics to help the sampler move smoothly through the space of possible answers. To do this, we introduce an extra variable called *momentum*, written as r . The mass matrix, written as M , tells us how this momentum turns into movement.

You can think of M as describing how “heavy” the sampler feels in each direction. If something is heavier, it moves more slowly; if it is lighter, it moves more easily. Most of the time, we simply use the identity matrix so every direction is treated in the same way.

Key Equations

Hamiltonian Energy

$$H(\theta, r) = U(\theta) + K(r)$$

This is the total “energy”.

- θ are the parameters we want to learn.
- r is the momentum, an extra helper variable.
- $U(\theta)$ is the potential energy, which depends on how likely the parameters are.
- $K(r)$ is the kinetic energy, linked to how fast the sampler moves.

Potential Energy

$$U(\theta) = -\log p(\theta \mid X, y)$$

This describes the “shape” of the problem.

- $p(\theta \mid X, y)$ is the posterior probability of the parameters.
- A lower $U(\theta)$ means a more likely value of θ .

Kinetic Energy

$$K(r) = \frac{1}{2} r^\top M^{-1} r$$

This controls how momentum turns into motion.

- r is the momentum.
- M is the mass matrix.
- M^{-1} means “the inverse of M ”.

If M is the identity matrix, then all directions have the same weight and the sampler moves evenly. If M is more complicated, it can help the sampler move more naturally when some directions behave differently from others.

13.4 Leapfrog Integration

HMC simulates Hamiltonian dynamics using the leapfrog integrator. Given a step size ϵ , the updates are:

1. Half-step momentum update

$$r_{t+\frac{1}{2}} = r_t - \frac{\epsilon}{2} \nabla_{\theta} U(\theta_t).$$

2. Full-step position update

$$\theta_{t+1} = \theta_t + \epsilon M^{-1} r_{t+\frac{1}{2}}.$$

3. Second half-step momentum update

$$r_{t+1} = r_{t+\frac{1}{2}} - \frac{\epsilon}{2} \nabla_{\theta} U(\theta_{t+1}).$$

These updates simulate the movement of a particle across the energy landscape defined by $U(\theta)$.

13.5 Metropolis Acceptance Step

After simulating Hamiltonian dynamics to propose a new state (θ', r') , the proposal is accepted with probability

$$\alpha = \min(1, \exp[H(\theta, r) - H(\theta', r')]).$$

If the move is accepted, the chain transitions to θ' . Otherwise, the chain remains at θ . This ensures detailed balance and guarantees that the Markov chain converges to the posterior distribution.

13.6 No-U-Turn Sampler (NUTS)

The No-U-Turn Sampler extends HMC by automatically choosing the number of leapfrog steps. NUTS stops the trajectory when it begins to double back on itself, preventing wasted computation and improving efficiency. NumPyro's NUTS implementation uses adaptive step-size selection and mass-matrix tuning during warmup.

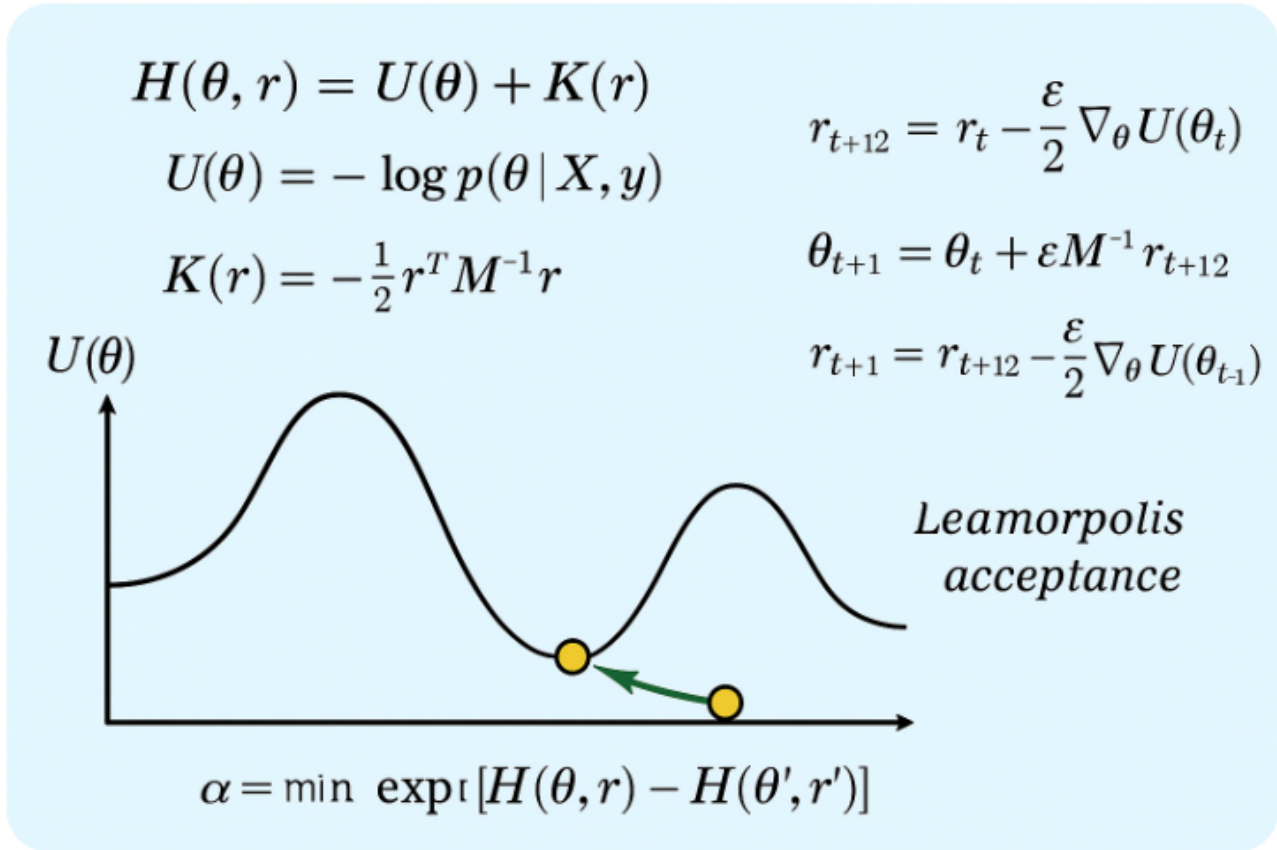


Figure 18: Example of theoretical landscape with energy and direction.

13.7 Hyperparameters

Table 8: MCMC Hyperparameters

Parameter	Value
Number of Warmup Samples	1000
Number of MCMC Samples	2000
Number of Chains	4
Number of Predictive Samples	500

13.8 Final Model Selection

TODO

13.9 Test Set Performance

14 Model Training and Development

14.1 Baseline Model

14.2 Error Analysis

[Analyse misclassifications or high-error predictions]

Figure 19: Error distribution for final model

15 Predictions and Deployment

15.1 Making Predictions

[Describe the prediction pipeline]

15.2 Model Deployment Strategy

[Outline how the model would be deployed in production]

- API design
- Infrastructure requirements
- Monitoring approach

15.3 Scalability Considerations

[Discuss computational requirements and scalability] Transition the JAX model to production environment (e.g., Google Cloud or specialised low-latency servers) using **optimised** serialisation formats for minimal inference time.

16 Model Training Results

17 Discussion

17.1 Interpretation of Results

[Discuss what the results mean in context of the problem]

17.2 Comparison with Hypothesis

[Relate findings back to your initial hypothesis]

17.3 Limitations

[Be honest about limitations of your approach and data]

- Data limitations: ...
- Model limitations: ...
- Computational constraints: ...

17.4 Potential Biases

[Discuss any biases in data or model]

References

References

[Chen and Guestrin, 2016] Chen, T. and Guestrin, C. (2016) XGBoost: A Scalable Tree Boosting System. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

[Fischer and Chen, 2018] Fischer, T. and Chen, C. (2018) Deep learning for more accurate stock price forecasting. *Journal of Financial Data Science*, 4(2), pp.17-31.

[Gillen and G. L., 2018] Gillen, G. and G. L. (2018) Hedonic Pricing and the Value of Aircraft Assets. *Journal of Air Transport Management*, 68, pp.178-185.

[Google Research, 2018] Google Research (2018) JAX: Composable transformations of Python+NumPy programs. [Online] Available at: <https://github.com/google/jax> [Accessed Date].

[Pflug and Römisch, 2007] Pflug, G. C. and Römisch, W. (2007) *Modeling, Measuring and Managing Risk*. World Scientific Publishing Co. Pte. Ltd.

Phan, D., Pradhan, N. and Jankowiak, M. (2019) Composable effects for flexible and accelerated probabilistic programming in NumPyro, arXiv.org. Available at: <https://arxiv.org/abs/1912.11554> (Accessed: 28 November 2025).